看不懂 int (*p)[3]？四要素法拆解一切复杂声明

Benjamin 2026-10-10 公众号文章 · 原链接 mp.weixin.qq.com

先看两个声明。它们只差一对括号：

```
int *p[3];
int (*q)[3];
```

不查资料，直接回答：`sizeof(p)` 和 `sizeof(q)` 各是多少？答案在 64 位平台上是 24 和 8。一对括号之差，类型完全不同：一个是"装指针的数组"，一个是"指向数组的指针"。看懂这对括号，就看懂了 C 语言所有复杂声明的读法，内核头文件里那些让人头皮发麻的原型，用的也是同一套规则。

一、一对括号，两种类型

先验证上面的大小，再各补一行解引用：

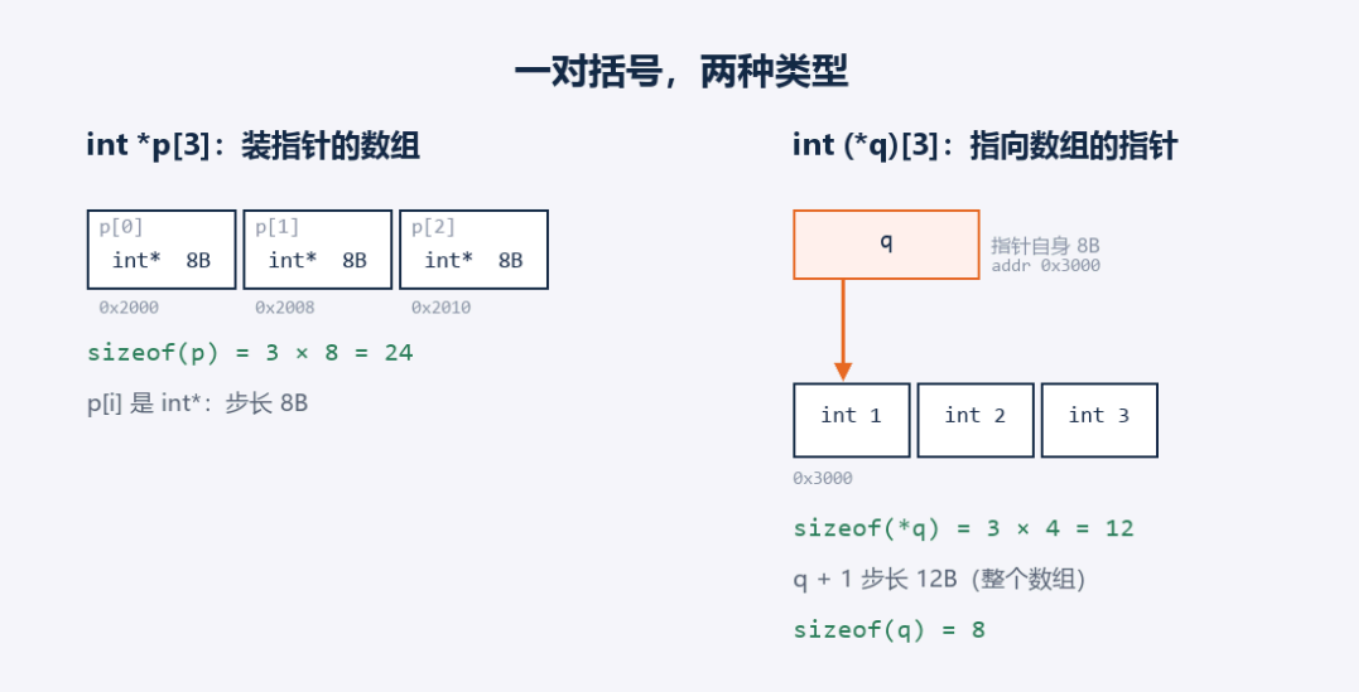
```
#include <stdio.h>

intmain(void)
{
    int *p[3];          // 数组：3 个 int* 元素
    int (*q)[3];        // 指针：指向 int[3]
    int arr[3] = {1, 2, 3};

    printf("sizeof(p)  = %zu\n", sizeof(p));    // 24
    printf("sizeof(q)  = %zu\n", sizeof(q));    // 8
    printf("sizeof(*q) = %zu\n", sizeof(*q));   // 12
    q = &arr;                   // q 指向整个数组
    printf("(*q)[1] = %d\n", (*q)[1]);          // 2
    return0;
}
```

三个数字各有含义。`p` 是数组：3 个元素，每个元素是 `int *`，占 3 × 8 = 24 字节。`q` 是指针：只有 1 个，占 8 字节；解一次引用 `*q` 得到的是 `int[3]`，12 字节。`p` 里能放 3 个地址，`q` 里只能放 1 个地址，但这个地址身后跟着一整块 12 字节的数组。

分水岭在哪？在 `[]` 和 `*` 的优先级。`[]` 的优先级高于 `*`，所以在 `int *p[3]` 里，`p` 离 `[3]` 更近，先被方括号抓住：`p` 是数组，剩下的 `int *` 描述每个元素的类型。而括号把 `(*p)` 圈起来，强迫 `p` 先与 `*` 结合：`q` 是指针，剩下的 `int [3]` 描述它指向什么。



图：`int *p[3]` 与 `int (*q)[3]` 的内存结构对比：一个是 3 格指针数组，一个是 1 个指针指向 12 字节的数组

同样一句话可以倒过来说：声明里的每个符号，描述的是"使用它时表达式长什么样"。这一条是读所有声明的总纲。

二、声明是使用方式的镜像

C 的声明语法有个著名设计原则：**声明模仿使用**（declaration mimics use）。`int (*q)[3];` 为什么长这样？因为使用 `q` 时，写 `(*q)[i]` 得到一个 `int`——把使用表达式里替换掉的部分换成类型 `int`，就是声明。数组、函数、指针混搭时，声明自然继承了表达式的优先级规则，所以 `[]`、`()` 压过 `*`，括号负责改变结合顺序。

K&R 第二版 §5.12 专门用一节讲"复杂声明"，甚至为此写了一个递归下降解析器来演示。这条原则还解释了一个经典陷阱：`int* a, b;` 里只有 `a` 是指针，`b` 是普通 `int`。`*` 修饰的是名字而不是类型，声明的类型逐个名字生效；把 `int*` 当成"一个类型"去理解声明，从这一行起就会处处踩坑。语法本身可以压缩成一张三步表：

步骤	操作
1. 找到名字	从最内层的标识符出发
2. 右左结合	先看右边：有 () 或 [] 就先结合；再看左边：吃掉 *；遇括号回到内部重来
3. 逐层释义	把每一层的结合结果翻译成一句自然语言

拆完之后，再用四个问题自查，把拆解结果落到使用规则上：

要素	对 q 的答案	决定什么
指针的类型	`int (*)[3]`	能赋值给谁、sizeof 结果
指针所指向的类型	`int[3]`	步长（12 字节）、解引用结果
指针的值	某个地址	解引用访问哪里
指针自身占的内存	8 字节（64 位）	与所指类型无关

其中"指针的类型"与"所指向的类型"这两个概念，是理解后面 `signal` 原型的关键：前者管指针本身怎么用，后者管步长和解引用。**四问里任何一问答不上来，就回到三步表再拆一层。**

三、实战：从 `int (*p)(int)` 读到 `signal` 原型

三步法上手，先拆两个常见类型：

声明	拆解	含义
`int (*p)(int);`	p 左结合 * → 指针；再右结合 (int) → 指向函数	函数指针，指向 int(int)
`int *(*f(int))[3];`	f 先右结合 (int) → 函数；*f(int) → 返回指针；再配 [3]	返回"数组指针"的函数

第二行是三步法的完整演练：`f` 右边有 `(int)`，先结合，得知 `f` 是函数；它左边的 `*` 说明返回值是指针；这个指针右边跟着 `[3]`，说明指向数组；数组元素是 `int *`。一层剥完再剥一

层，每层用一句自然语言记录，剥到最外层就读完了。

现在上真正的"天书"，UNIX 信号函数的原型：

```
void (*signal(int sig, void (*handler)(int)))(int);
```

按三步法走一遍。第一步，找到名字 `signal`。第二步，`signal` 右边是 `(int sig, void (*handler)(int))`，先结合：`signal` 是函数，两个参数，一个 `int`，一个"指向 `void(int)` 函数的指针"。第三步，去掉参数部分，剩下 `void (*...)(int)`：返回值也是"指向 `void (int)` 函数的指针"。合起来一句人话：`signal` 注册一个信号处理函数，返回旧的处理函数。语义一句话，原型层层嵌套。



图：signal 原型的三步拆解过程：从名字出发，先右结合参数，再逐层剥出返回类型

工程上遇到这种原型，第一反应不是硬读，而是 `typedef` 化简：

```
typedef void (*sighandler_t)(int);          // 先给"处理函数指针"起名

sighandler_t signal(int sig, sighandler_t handler);
```

两行读完，和原型那一行完全等价。glibc 的 `signal.h` 正是这么写的：头文件里先 `typedef void (*__sighandler_t) (int);`，再声明 `signal`——所以库源码比你想象的易读，难读的只是被迫单行书写的原型。

声明语法难读这件事，连语言作者自己都承认。Bjarne Stroustrup 曾把 C 的声明语法称作"一场失败的实验"（an experiment that failed）；K&R 更早就在书里写了解析器来对付它。读内核头文件迟早会撞上这类声明，好在规则只有三步：找名字、右左结合、逐层释义，剩下的只是耐心。

写在最后

把这篇的方法压成五条，可以当检查表用：

1. 判断第一刀：看名字右边有没有 `[]` 或 `()`，有就先结合，没有才轮到左边的 `*`。`int *p[3]` 是数组，`int (*p)[3]` 是指针，括号的作用就是改变结合顺序。
2. 拆完用四问自查：类型、所指向的类型、值、自身大小。步长和解引用结果由"所指向的类型"决定，别把两个"类型"混为一谈。
3. 遇到多层嵌套不要慌：从名字出发一层层剥，每剥一层用一句自然语言记录，剥到最外层声明就读完了。
4. 工程代码里读到难读的原型，先 `typedef` 化简再使用；glibc 头文件里的写法就是现成范例。
5. `const` 也吃这套规则：从名字出发右左结合，`char *const p` 是"指针自身只读"，`const char *p` 是"所指内容只读"，读法与本文完全一致。

下一篇我们聊聊多级指针的算术：`char **ptr` 自增一次，地址到底跳多少？两个流行却错误的推演方式坑过无数人，四要素法能一眼看穿它们错在哪。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。